



Course Name

INTRODUCTION TO PROBLEM SOLVING

FULL COURSE SUMMARY

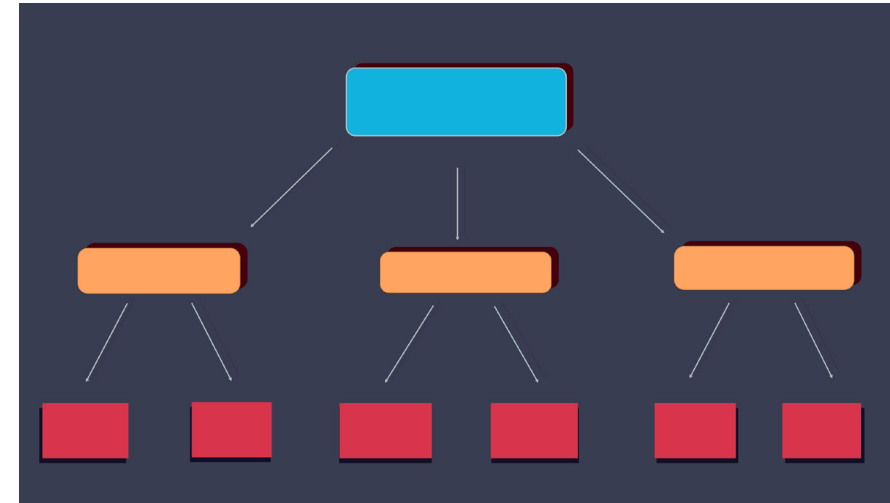
IN SUMMARY

Firstly, you learned the importance of thoroughly understanding problems before solving them and how to create effective plans for solutions. You also learned about algorithms, pseudocode, and flowcharts as tools for problem-solving, as well as the importance of problem decomposition, algorithmic thinking, and planning skills for real-world applications. Next, you learned about problem-solving strategies, focusing on the importance of choosing the right approach for different challenges. We examined four strategies: brainstorming, heuristics, trial-and-error, and means-ends analysis, discussing their applications and considerations. It was emphasised that understanding these strategies is essential for developing effective and efficient solutions to a wide range of problems. Subsequently, you learned about problem-solving approaches. We focused on breaking down complex problems into simpler parts and also on how to prioritise immediate, optimal choices. We covered both greedy algorithms and divide and conquer as effective problem-solving techniques. Lastly, you learned the fundamental concepts of algorithms, with an emphasis on their role as ordered, unambiguous sets of instructions for problem-solving. The

characteristics of effective algorithms, their importance in improving performance and scalability, and the factors to consider during their design were also discussed.

BULLET POINT SUMMARY

- It is important to understand the significance of problem comprehension before solving it.
- This involves learning about algorithms, pseudocodes, and flowcharts, which are used for creating solution plans.
- Problem decomposition is a key strategy that should be emphasized for effective problem-solving.
- Algorithms are procedures that help solve problems in a structured way. Pseudocodes, on the other hand, are textual representations of algorithms that enable us to better understand the logic behind them.
- Flowcharts are graphical representations of algorithms that help us visualize the steps involved in solving a problem.
- Practising problem-solving skills systematically is encouraged to improve your problem-solving skills.
- The divide and conquer technique is a problem-solving approach that involves breaking down complex problems into simpler parts. #



```

    if lang is None:
        raise Exception("Input language could not be determined")
    parsedInput = self.parseInputToLanguageModel(inputString, inputLanguage, context)
    if not parsedInput or not self.model:
        return None
    context.append(parsedInput) # Add new conversation entry to context
    return (self.model.generateLLMOutput(parsedInput), context)

    def parseInputToLanguageModel(inputString, inputLanguage, context):
        if self.model is None or self.model.language != inputLanguage:
            # LLM is not initialised or has wrong language, load LLM
            self.model = self.loadAllLanguageModelFromDatabase(inputLanguage)
            if self.model is None or not self.runModelSelfDiagnosis():
                raise Exception("AI language model load failed")
            return None
        self.model.setLLMContext(context) # Put past conversation context into LLM
        llmInputParser = self.model.getInputParser()
        return llmInputParser.parseInput(inputString)
  
```

- For it to be effective, certain conditions have to be met. On the other hand, greedy algorithms are those that prioritize immediate optimal choices when solving problems.
- However, it's important to note that this approach may not always provide the best possible solutions.
- Key characteristics of algorithms include input, unambiguity, finiteness, effectiveness, and language independence.
- Algorithms are important for achieving better performance and scalability. When designing an algorithm, there are several factors to consider such as modularity, correctness, maintainability, functionality, robustness, simplicity, and extensibility.

CASE STUDY

CASE STUDY 1

TITLE: THE DECISION-MAKING DILEMMA AT XYZ CORPORATION

Andela Corp tackles a crucial challenge—deteriorating product quality affecting sales and customer satisfaction. The objective is swift resolution through problem-solving strategies:

- Root Cause Analysis: Identify manufacturing process issues.
- Data Analysis: Utilise customer feedback, quality records, and production reports.
- Cross-Functional Team: Form a diverse team for varied insights.
- Quality Reinforcement: Strengthen control, optimise processes, and collaborate with suppliers.
- Employee Training: Invest in training and establish a customer feedback loop.
- Continuous Improvement: Foster a culture of ongoing enhancement with transparent

CASE STUDY 2

TITLE: OPTIMISING DELIVERY ROUTES FOR A FOOD DELIVERY SERVICE

In the case study, a city-based food delivery company seeks solutions for route optimization to minimize delivery times and fuel costs. The challenge prompts the exploration of strategies like greedy algorithms or divide and conquer to enhance the efficiency of its delivery operations in a bustling urban environment.



Key references:

Bransford, J. D., & Stein, B. S. (1993). The ideal problem solver
Treffinger, D. J., Isaksen, S. G., & Stead-Dorval, K. B. (2006). Creative

QUESTIONS TO PONDER

- Why is understanding the problem thoroughly considered a crucial step in problem-solving? How can this concept be applied to real-life situations?
- What are the key differences between algorithms, pseudocodes, and flowcharts, and how do they aid in structuring and solving problems effectively?
- What are the four problem-solving strategies discussed in the text, and why is it important to understand them?
- Can you explain the key principles of effective brainstorming, as described in the text? How can these principles contribute to successful problem-solving?
- Explain the fundamental principle of the divide and conquer technique. How does it break down complex problems, and what are the key conditions for its effectiveness?
- Provide an example outside of computer science where divide and conquer can be applied to solve a problem. Describe the problem and how the technique would be implemented.
- How might the principles of algorithm design apply to everyday decision-making processes, beyond just computer programming?
- Can you think of any scenarios where an algorithm's lack of robustness could lead to unintended consequences or even ethical concerns?

SKILLS AND COMPETENCIES YOU HAVE ACQUIRED AFTER THIS LESSON

Problem-solving skills, Algorithmic thinking, Planning and organisation, Understanding complex problems, Breaking problems into manageable parts, Time Management, Problem Awareness, Algorithm design, Time and resource management.

PERSONAL REFLECTION

Reflect on a recent challenge you faced and how understanding the problem and creating a solution plan could have improved the outcome. Consider how algorithms, pseudocodes, and flowcharts can be applied in your daily life to tackle complex issues more effectively.



Key references:

Muniswamy, V. V. (2013) Design and Analysis Of Algorithms IK International Pvt Ltd.
"Human Powered Flight Group Webpage". Royal Aeronautical Society, 2011. Retrieved January 18, 2021.

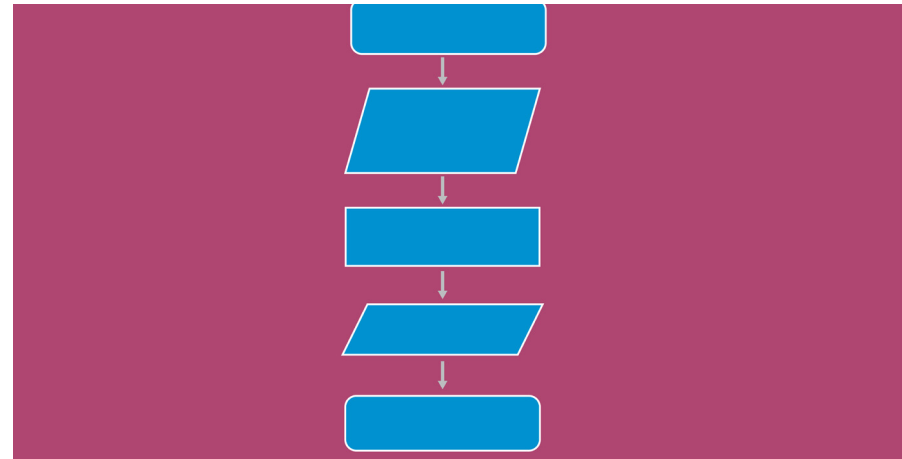
IN SUMMARY

Firstly, you learned about flowchart concepts, which began with an introduction to flowcharts as graphical representations of computer programme operations. You also learned about the history of flowcharts and their varied applications, emphasising their role in simplifying complex processes and improving information display. Additionally, you learned about the key flowchart symbols such as process, start/stop, and decision, explaining their significance. We also outlined the advantages of using flowcharts, such as clarifying programme logic and facilitating communication among stakeholders, while also addressing their limitations, such as the potential for complexity. Next, you learned about the concepts of pseudocode. You also discovered that pseudocode acts as a bridge between English language and programming. The lesson also highlighted the importance of creating clear and language-independent pseudocode and explained four key rules to follow. In addition, the lesson discussed the benefits of pseudocode,

such as its accessibility and flexibility, as well as its potential drawbacks, like non-standardised formats and reduced visual clarity compared to flowcharts. Subsequently, you learned about the concept of programme objects and classes in computer programming. We went over how real-world objects have state and behaviours, which is reflected in programming through variables and methods. We also stressed the significance of data encapsulation in classes and how access specifiers are used to manage access to class members. Additionally, we discussed the advantages of object-oriented programming, such as code reusability and modularity. Lastly, you learned about abstraction, functions, and modules in programming. Abstraction simplifies complex systems, allowing users to interact only with high-level functionalities. Functions and modules break down complex projects, enabling multiple contributors, fostering code reusability, and making it easier to manage large and complex projects.

BULLET POINT SUMMARY

- Flowcharts are graphical representations of computer program operations.
- Key symbols used in flowcharts include process, start/stop, and decision symbols.
- Pseudocode is a concept used for making algorithms more readable. It is a transition from flowcharts towards a more structured code-like format.
- The purpose of pseudocode is to make the algorithm more accessible and flexible.
- There are some disadvantages of pseudocode, such as non-standardized formats, which can reduce visual clarity.
- Data encapsulation is a programming concept that refers to the practice of hiding the implementation



details of an object from the user. This means that the user can only interact with the object through a set of public methods, while the internal workings of the object remain hidden.

- The benefits of object-oriented programming include modularity, code reusability, and the independence of objects.
- Objects can be classified in programming, and access specifiers like public and private control the access to class members.
- Functions are logical blocks with defined interfaces for task execution, shielding implementation details.
- Modules encompass multiple functions and classes, suitable for large-scale projects.
- Functions and modules simplify complex projects, enable collaboration, and promote code reusability in software development.

CASE STUDY

CASE STUDY 1:

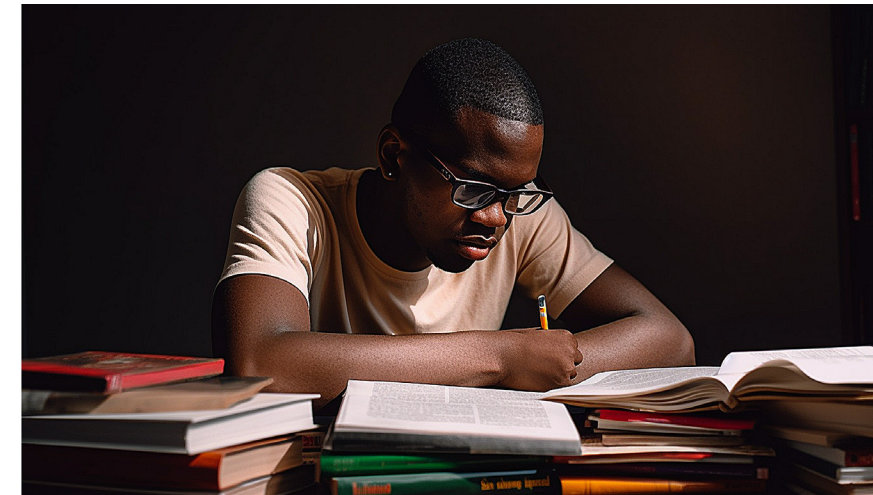
TITLE: ENHANCING PROGRAMME LOGIC CLARITY WITH FLOWCHARTS

The case study explores the benefits of employing flowcharts in software development for improved program logic and communication with non-technical stakeholders. The methodology involves systematic flowchart creation, defining symbols, mapping logic step-by-step, breaking down complex tasks, and incorporating error-handling paths. Regular reviews, stakeholder feedback, and comprehensive documentation ensure clarity, while iterative refinement adapts the flowchart to evolving requirements for optimal software.

CASE STUDY 2:

TITLE: CLASSES AND PROGRAMME OBJECTS IN LIBRARY SOFTWARE

The objective of this case study is to create an efficient and secure library management system using programme objects and classes. The emphasis is on designing a class structure that ensures effective data encapsulation for book management, with a focus on data security. The methodology involves devising a robust class structure with appropriate access specifiers to enhance library operations and provide a secure platform for efficient book management in the digital age.



QUESTIONS TO PONDER

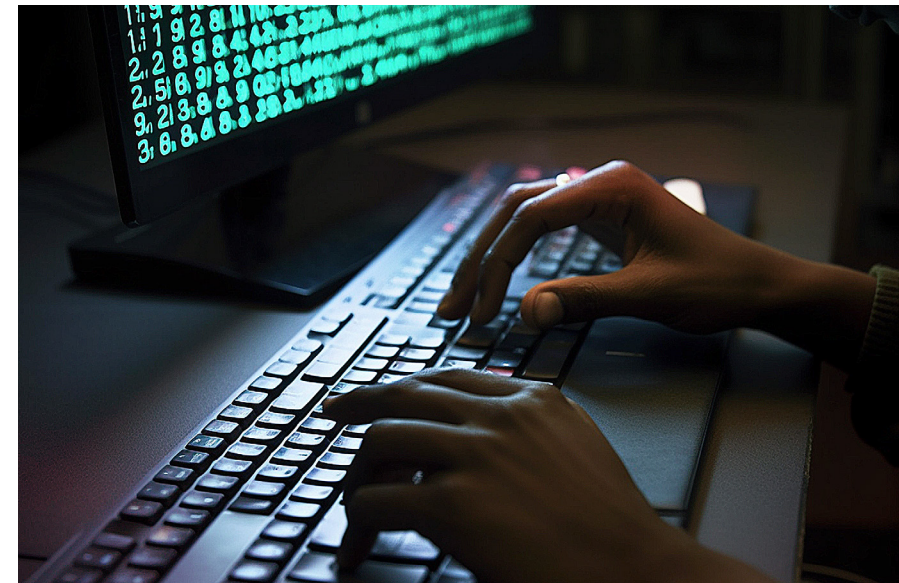
- How have flowcharts evolved over time, and what is their significance in contemporary computer science and information display?
- What are the fundamental components of a flowchart, and how do they contribute to representing the sequence of operations in a computer programme?
- Why do we need pseudocodes when we already have flowcharts for representing algorithms?
- How does the use of pseudocodes bridge the gap between conceptualising an algorithm and actual programming?
- What are some real-world scenarios where the concept of breaking down a complex problem into smaller problems is applicable, and how can object-oriented programming be used to address the problem more effectively?
- How can you apply the principles of abstraction to simplify complex software systems and enhance their maintainability?

SKILLS AND COMPETENCIES YOU HAVE ACQUIRED AFTER THIS LESSON

Understanding of flowchart concepts, Symbol interpretation, Creating flowcharts, Problem-solving, Communication, Critical thinking, Pseudocode writing, Clarity in communication, Collaboration, Technical writing, Visual interpretation, and Adaptability.

PERSONAL REFLECTION

How has your perspective on data encapsulation and access specifiers in object-oriented programming evolved, and in what ways do you envision applying this newfound understanding to enhance the security and integrity of your code?



Key references:

Chaudhuri, A. B. (2020). Flowchart and algorithm basics: The art of programming. Mercury Learning and Information.

IN SUMMARY

Firstly, you learned about the fundamental concepts of programme control and branching in computer programming. We emphasised the importance of sequential execution and how branching alters the flow of program control, covering both unconditional branching with the 'goto' statement and conditional branching using decision-making statements such as 'if,' 'if-else,' and 'if-else-if.' We further explained the essential elements of programming, focusing on statements and blocks. We distinguished between simple statements, each ending with a semicolon, and compound statements enclosed within curly brackets. You learned that blocks are crucial for solving complex problems by breaking them into smaller, more manageable units.

Next, you learned about the concept of recursion in computer science. Recursive functions break down a problem into smaller subproblems and merge their solutions. A function calls itself in recursion, with an exit condition to prevent infinite calls. We explained step-by-step how a recursive function calculates its final result, emphasising the importance of this concept in solving computational problems. Subsequently, you learned about the fundamental concepts of decomposition and modularization in the field of programming. The main idea is to break down complex problems into smaller, manageable parts using independent modules and functions. This approach makes the code more understandable, maintainable, and easier to manage. We also discussed the importance of module independence, emphasising the single responsibility principle, and explained cohesion and coupling as key factors in measuring it. Lastly, you had an extensive overview of software testing, both the software testing life cycle and various testing methods. We discussed the importance of structured testing processes to ensure software reliability. The software testing life cycle phases, from requirement analysis to test cycle closure, are elaborated upon. We then looked at software testing methods, categorising them into functional and non-functional types. Functional methods, including unit, integration, system, and acceptance testing, ensure software performs its intended functions. Non-functional methods, such as usability, performance, security, and compatibility testing, focus on non-functional aspects like user experience, security, and compatibility across various environments.

BULLET POINT SUMMARY

- Blocks are utilized to break down complex problems into manageable units, creating simple statements from compound statements.
- Recursive functions are used to solve problems that require repetitive calculations. When a function calls itself, it creates a new instance of the function with a new set of parameters.



Key references:

Kernighan, B. W., & Ritchie, D. M. (2002). The C programming language.
Bailey, T. (2005). An introduction to the c programming language and software design. July-2005

- Each instance of the function executes until it meets the exit condition, at which point it returns a result to the previous instance. The previous instance then continues executing until it also meets the exit condition and returns a result to its previous instance.
- Recursion is important in solving complex computational problems because it allows for the creation of more elegant and concise solutions. It also enables the use of divide-and-conquer algorithms, which can greatly improve the efficiency of certain algorithms.
- Cohesion and coupling are key factors in measuring module independence.
- Software Testing Life Cycle consists of six phases.

CASE STUDY

CASE STUDY 1:

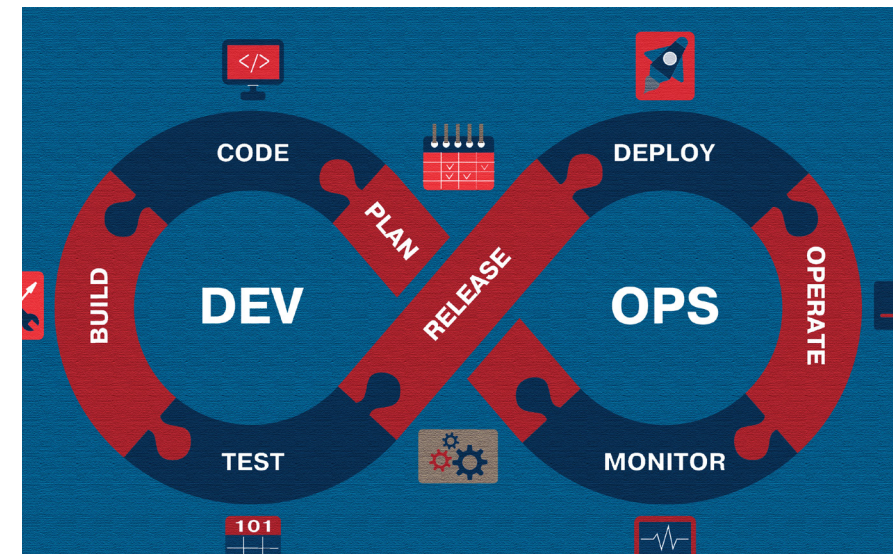
TITLE: A RECURSIVE APPROACH TO PACKING A SUITCASE

This case study explores the application of recursion in a relatable context: packing a suitcase efficiently. By introducing the `pack_suitcase()` function, taking the suitcase and item as parameters, the recursive approach maximizes space utilization. The function checks if the item fits; if not, it recursively calls itself a smaller version of the item. This demonstrates the power of recursion in solving practical problems, enhancing understanding through the relatable scenario of efficient suitcase packing.

CASE STUDY 2:

TITLE: USING A WELL-DEFINED STLC TO ENSURE SOFTWARE QUALITY

This case study delves into the application of software testing in software development. Software testing is integral in ensuring software meets requirements. The Software Testing Life Cycle (STLC) guides this process systematically. In a scenario involving a mobile banking app, a team outlines requirements and prepares for testing. They begin with a test plan, defining scope, tests, and resources. Designing specific test cases, executed and evaluated for defects, and reported to the development team for resolution.



Key references:

Kernighan, B. W., & Ritchie, D. M. (2002). The C programming language.
Bailey, T. (2005). An introduction to the c programming language and software design. July-2005

QUESTIONS TO PONDER

- How does the concept of programme control impact the execution of a computer programme, and why might you need to alter this control flow?
- In what situations would unconditional branching, such as the 'goto' statement, be a useful tool in programming, and are there any potential drawbacks or best practices to consider?
- How can recursion be used to solve real-world problems?
- What are some alternative approaches to solving problems that can be solved with recursion?
- How can I use modularization to improve the efficiency and maintainability of my project?
- How can software testing be used to support the success of digital transformation initiatives?

SKILLS AND COMPETENCIES YOU HAVE ACQUIRED AFTER THIS LESSON

Programming fundamentals, Branching, Statement and block usage, Code readability, Problem decomposition, Decision-making, Application of concepts, Real-world problem-solving, Non-functional testing, Critical thinking, Communication and reporting, Quality assurance, Adherence to process.

PERSONAL REFLECTION

How can I apply the concepts and methodologies from software testing to enhance the quality of my work or projects, irrespective of the field?

In reflection, how has my comprehension of recursion in programming transformed through study? Can I envision practical scenarios where employing recursion would address intricate problems or optimize my code's efficiency?



Key references:

Kernighan, B. W., & Ritchie, D. M. (2002). The C programming language.
Bailey, T. (2005). An introduction to the c programming language and software design. July-2005